

Gravity Systems for Multi-Oriented Environments

Piper Inns Hall

This report examines dynamic gravity direction systems—a game-play technology enabling players to manipulate gravitational orientation in real-time. The implementation combines quaternion-based rotation mathematics, cardinal direction snapping, and smooth animation blending to create intuitive, responsive gravity-switching mechanics. The system enables novel level design possibilities in puzzle and platformer games, drawing inspiration from titles like *Manifold Garden*, *Disoriented*, *Portal*, and *Mario Galaxy*.

July 24, 2026

CGRA 359: Games and Graphics Project
Assignment 1 – Technology Demonstration
Victoria University of Wellington
[Video Demonstration](#) °

Contents

1. The Problem	4
1.1. Game Development Challenge	4
1.2. Limitations of Previous Approaches	4
1.3. Relevance to Modern Game Development	4
2. The Technology	5
2.1. Technical Background and Evolution	5
2.2. Core Implementation Details	5
2.2.1. Player Physics	5
2.2.2. Cardinal Snapping	5
2.2.3. Gravity Manager	5
2.2.4. Gravity Zones	5
2.3. Practical Applications in Game Development	5
2.4. Optional Variations and Advanced Techniques	6
3. Implementation Analysis	7
3.1. Demonstration Overview	7
3.2. Technical Challenges Encountered	7
3.3. Performance Considerations	7
3.4. Integration with Other Game Systems	7
3.5. Code Quality	7
4. Industry Usage	8
4.1. Current Adoption in the Game Industry	8
4.2. Specific Use Cases and Examples	8
4.3. Why Developers Choose This Over Alternatives	8
5. Future Outlook	9
5.1. Technology Trajectory and Evolution	9
5.2. Potential Replacements or Improvements	9
5.3. Career Relevance for Game Developers	9
6. Resources	10
6.1. Effective Search Terms for Further Research	10
6.2. Link to Authoritative Explanation	10
6.3. Link to Practical Tutorial	10
6.4. Additional Learning Resources	10
Bibliography	11

1. The Problem

1.1. Game Development Challenge

This technology addresses the challenge of implementing non-standard gravity and player movement systems in games. Traditional engines assume gravity acts in one fixed direction, limiting designs involving walls, ceilings, curved surfaces, or different gravity orientations.

The system allows gravity direction to change dynamically while maintaining player control, collision handling, camera orientation, and movement physics. It provides a framework for gravity-based mechanics without manually rotating levels or creating separate movement systems.

1.2. Limitations of Previous Approaches

Previous gravity systems are often limited by their intended gameplay. Games like *Manifold Garden* use gravity manipulation for puzzles but restrict when and where gravity changes occur.

Disoriented uses gravity through specific level designs rather than a general framework. *Super Mario Galaxy* uses planetary gravity effectively but relies on momentum-based movement, making it less suitable for precision gameplay.

This technology provides a flexible gravity system supporting responsive movement and varied level design.

1.3. Relevance to Modern Game Development

This technology enables flexible approaches to player movement and level design. As games experiment with unique traversal mechanics and physics-based puzzles, dynamic gravity systems allow experiences that are difficult to achieve with traditional movement systems.

A customizable gravity system supports wall-walking, environmental puzzles, spatial exploration, and alternative navigation.

2. The Technology

2.1. Technical Background and Evolution

As games explored unusual movement mechanics, developers created custom gravity systems that allow the player's reference frame to change. These systems are used for planetary movement, wall-walking, and gravity-based puzzles.

Modern approaches treat gravity direction as a gameplay variable rather than a fixed engine value. This allows developers to modify player orientation and movement while continuing to use standard physics.

Research into existing gravity systems, including discussions of gravity zones and transform-based player orientation in [1], reinforced separate gravity handling and player rotation updates.

2.2. Core Implementation Details

The system consists of three components: player physics, gravity management, and gravity zones.

2.2.1. Player Physics

When gravity changes, the player's physics frame updates by calculating a rotation between the old and new up directions. The visual frame is separate and uses SLERP interpolation to smoothly rotate the camera and character model.

2.2.2. Cardinal Snapping

Gravity flips are limited to four directions based on camera orientation. A threshold prevents flips when the camera is between directions.

2.2.3. Gravity Manager

The gravity manager calculates new directions, applies rotations, and manages player state during transitions. It also tracks active gravity zones and disables manual flips while inside them.

2.2.4. Gravity Zones

Gravity zones create localized gravity fields using an origin point. They can pull the player toward the origin or push them away.

2.3. Practical Applications in Game Development

Dynamic gravity systems support wall-walking, planetary movement, spatial puzzles, and alternative traversal systems. These mechanics allow levels that do not rely on traditional horizontal surfaces.

Gravity manipulation can create navigation challenges, unusual platforming mechanics, and experimental designs involving non-Euclidean spaces.

2.4. Optional Variations and Advanced Techniques

Advanced gravity systems can expand beyond fixed directional changes using more complex gravity fields. Variations include spherical planetary gravity, overlapping gravity zones, and gameplay-driven gravity changes.

Additional techniques include blending gravity sources, applying custom forces, and separating physics, visuals, and camera systems.

3. Implementation Analysis

3.1. Demonstration Overview

The demonstration is built in Godot and showcases a dynamic gravity system with manual gravity switching and gravity zones. The player can rotate gravity while maintaining responsive movement and smooth visual transitions.

3.2. Technical Challenges Encountered

The main technical challenge was achieving accurate rotations during gravity changes. Managing the player hierarchy was difficult because the player used a harness, pivot, and camera system. The mesh had to be attached to the harness to prevent snapping.

I considered removing the rotation animation because it introduced unwanted momentum, but kept it for visual quality. This required separating physics rotation from visual interpolation.

3.3. Performance Considerations

The system minimises unnecessary processing by only updating gravity when the active gravity zone changes direction. The gravity zone manager checks player position each frame but only applies rotation calculations when the angle difference exceeds a threshold.

This avoids repeated quaternion calculations while maintaining smooth gravity behaviour.

3.4. Integration with Other Game Systems

The gravity system integrates with the player controller by modifying the player's up direction while preserving movement, jumping, and collision behaviour. Movement is calculated relative to the current gravity orientation.

3.5. Code Quality

The gravity system separates physics, gravity management, and visual rotation. Complex calculations are commented while simple plumbing remains concise.

4. Industry Usage

4.1. Current Adoption in the Game Industry

Dynamic gravity systems are used in games requiring unconventional movement, spatial puzzles, or alternative level design. They are usually created as custom gameplay systems rather than engine features.

4.2. Specific Use Cases and Examples

Games such as Manifold Garden use gravity changes as a puzzle mechanic, allowing players to rotate perspectives and navigate impossible spaces. Super Mario Galaxy uses localized planetary gravity for unique platforming experiences.

4.3. Why Developers Choose This Over Alternatives

Developers use custom gravity systems because they provide more freedom than fixed-axis physics. Gravity becomes a gameplay mechanic enabling new movement styles, puzzle solutions, and level structures.

5. Future Outlook

5.1. Technology Trajectory and Evolution

Gravity systems are likely to become more flexible as physics engines improve. Future implementations may support complex gravity fields, smoother transitions, and procedural worlds.

5.2. Potential Replacements or Improvements

Future alternatives may use advanced physics frameworks or engine-level support for custom gravity. Improvements could include automatic player orientation handling and more efficient calculations.

5.3. Career Relevance for Game Developers

Understanding custom gravity systems provides experience with physics, transforms, quaternion mathematics, and player controllers. These skills apply to gameplay programming and technical design.

6. Resources

6.1. Effective Search Terms for Further Research

- Dynamic gravity systems in games
- Quaternion player rotation
- Gravity zones
- Godot CharacterBody3D controller
- Gravity flip games

6.2. Link to Authoritative Explanation

- Godot Engine documentation:
 - <https://docs.godotengine.org/>[◦]
 - https://docs.godotengine.org/en/stable/classes/class_quaternion.html[◦]
 - https://docs.godotengine.org/en/stable/classes/class_vector3.html#vector3[◦]

6.3. Link to Practical Tutorial

- [2] demonstrates rapid level prototyping using Godot's built-in CSG tools.
- [3] demonstrates the structure and implementation of a first-person controller in Godot.

6.4. Additional Learning Resources

- Quaternion rotation and interpolation techniques
- Transform bases and coordinate spaces in 3D engines
- Physics-based character controllers

Bibliography

- [1] StayAtHomeDev, "My Gravity-Defying Retro FPS." [Online]. Available: <https://www.youtube.com/watch?v=Tix8SVsKUtg>^o
- [2] Garbaj, "Godot's Hidden Level/Map Editor." [Online]. Available: <https://www.youtube.com/watch?v=BUjCtwLO0S8>^o
- [3] Aurenel, "First Person Controller Tutorial - Godot 4.6." [Online]. Available: <https://www.youtube.com/watch?v=PLACEHOLDER>^o